

Isaac Hermel Reginato

isaachermel@gmail.com | +55 51 99709-9876 | Porto Alegre, RS (Remoto/Híbrido) | linkedin.com/in/isaachermel | github.com/h3rmel | isaachermel.dev.br

RESUMO PROFISSIONAL

Engenheiro de Software full-stack com 4 anos de experiência no ecossistema JavaScript/TypeScript. Mantenedor de biblioteca open-source com ~102 stars distribuída via registro shadcn (guarahooks — 50 React hooks) e contribuidor de design system em produção (StencilJS/Storybook). Atuação de ponta a ponta: front-end em React/Next.js com foco em performance (Core Web Vitals) e back-end em NestJS/Node.js, Python e .NET com APIs REST, pipelines ETL orquestrados (Apache Airflow) e integrações sob rate limit, em arquitetura em camadas com injeção de dependências. Autoria de biblioteca interna consumida por quatro aplicações e condução de DOJOs técnicos para disseminação de padrões. Estratégias de teste Shift-Left (TDD/BDD) e entrega sob feature flags.

PRINCIPAIS COMPETÊNCIAS

System Design | Arquitetura em Camadas | Injeção de Dependências | Design de Bibliotecas Internas | Performance / Core Web Vitals | SSR/SSG | Shift-Left Testing (TDD/BDD) | APIs REST | Mensageria & Processamento de Dados | Design Systems | CI/CD

HABILIDADES TÉCNICAS

- **Linguagens:** JavaScript, TypeScript, Python, C#;
- **Front-end:** React.js, Next.js, Vue, Blazor, Tailwind CSS, Framer Motion, Bootstrap, PWA offline-first;
- **Estado & Dados (front):** Zustand, Context API, Axios, consumo de APIs RESTful, comunicação em tempo real (WebSockets/SSE);
- **Design Systems:** Storybook, StencilJS (Web Components), registro de pacotes shadcn, content-collections;
- **Back-end & Integrações:** Node.js, Express.js, NestJS, Flask, .NET; fluxos de autenticação (OAuth 2.0, JWT); pipelines de ingestão e processamento de dados (ETL), Apache Airflow, orquestração de ETL;
- **Bancos de Dados & Mensageria:** PostgreSQL, MongoDB (modelagem de schema e migrações), Redis, Kafka;
- **Cloud, DevOps & Observabilidade:** AWS (S3, CloudFront), GCP (Firebase, Cloud Run), Kubernetes (manifests de deploy dos serviços sob ownership), Docker, Argo CD, GitHub Actions, GitLab CI/CD, Grafana (integração de alertas);
- **Testes:** Vitest, Jest, Testing Library; metodologias TDD e BDD;
- **Arquitetura & Práticas:** SOLID, arquitetura em camadas, injeção de dependências, Result Pattern, TDD, BDD, MVC, MVVM;
- **Ferramentas:** Git (GitFlow), Figma; desenvolvimento assistido por IA com workflow spec-driven (OpenSpec), regras e specs versionadas em repositório — aplicado em três repositórios públicos (guarahooks, enterprise-react-architecture, distributed-systems-lab).

EXPERIÊNCIA PROFISSIONAL

Engenheiro de Software, Zenvia — 08/2023 – Presente

- **Ownership (Back-end):** Ownership do `partners-api` — serviço de importação de dados em lote via leitura, extração, tratamento e processamento de arquivos CSV, estruturado em arquitetura em camadas com injeção de dependências para isolar regras de negócio das fronteiras de I/O; sustentando picos de 60 mil linhas/minuto em produção;

- **Ownership (Integrações & Dados):** Ownership dos pipelines ETL de sincronização com ERPs (Bling, Tiny, Microvix) — DAGs em Apache Airflow (Python) cobrindo contatos, pedidos, notas fiscais e catálogo de produtos, com carga inicial total e sincronizações incrementais. Arquitetura desenhada contra dois orçamentos aninhados da API do fornecedor (3 req/s e 120 mil req/dia, por cliente) em cenário de N+1 sobre HTTP: escalonamento work-conserving com alocação ponderada por prioridade entre fluxos concorrentes e retomada durável por checkpoint ao atingir a cota diária. Ownership assumido após a contribuição de otimização;
- **Ownership (Bibliotecas internas):** Autoria do `security-settings-sdk` — cliente HTTP autenticado com camada de cache para o serviço de configuração antiphishing, consumido por quatro aplicações (URL Manager, Templates Management, api-control e api-subscription-control), eliminando a duplicação de lógica de cliente e cache entre os repositórios. Biblioteca deliberadamente fina: não instancia infraestrutura, recebe o cache por porta injetada e os orçamentos de timeout do chamador, e não decide fail-open/fail-closed — reporta a condição e mantém a política visível no código consumidor. Erros expostos como união discriminada tipada (Result Pattern), separando falha de upstream, de autorização e de contrato; cache com TTL duplo e revalidação síncrona, deduplicação de chamadas concorrentes por chave, e invalidação por delete em vez de write-through, por ausência de garantia de ordem entre quatro consumidores do mesmo tópico Kafka;
- **Biblioteca de logger (Node.js):** Correção de regressão de identidade de instância na biblioteca de logging da organização — a exportação por object spread rompia a cadeia de protótipos `winston.Logger` → `stream.Transform` → `events.EventEmitter`, quebrando integrações de terceiros, verificação por `instanceof`, isolamento de contexto em testes e mocking. Substituição por patching estruturado sobre a instância nativa, com interceptação de `.log()` para validação, imutabilidade e propagação de contexto via `AsyncLocalStorage`, preservando referência ao método original contra recursão em espionagem. Ampliação das suítes unitária e de integração para cobertura total;
- **Dados & Operação (mensageria WhatsApp):** Coordenação da atualização de schemas e da migração das coleções de WhatsApp em MongoDB para incorporação dos novos identificadores BSUID (Business-Scoped User ID) da Meta, gerados para identificar usuários na API do WhatsApp Business. Conexão do Grafana ao Google Chat como canal de alerta da operação, roteando alertas de métricas de plataforma e de negócio — lag de consumidor Kafka, abertura e fechamento de contas WhatsApp e suspeita de phishing — para notificação acionável ao time;
- **Disseminação & Governança técnica:** Condução de DOJOs para o time — Result Pattern, adotado em seguida como padrão de tratamento de erro no `security-settings-sdk`, e comparação instrumentada entre Winston e PinoJS com testes e resultados medidos, embasando a decisão sobre nova biblioteca de logging. Padronização da geração de documentação das bibliotecas internas com JSDoc e TypeDoc, produzindo README consistente por biblioteca, estruturado para consumo por desenvolvedores e por ferramentas de IA; templates de documentação e padrão de changelog adotados em múltiplos projetos; Prettier e ESLint obrigatórios no pipeline GitLab CI;
- **Front-end (Design System & Performance):** Contribuição para o design system da empresa em StencilJS (Web Components), com Storybook para desenvolvimento e documentação isolada de componentes. Otimização de Core Web Vitals em produção (métricas de campo): LCP reduzido de 2.91s para 2.35s e INP de 275ms para 199ms, ambos movidos da faixa "precisa melhorar" para "bom", via refatoração da renderização em React.js com padrão MVVM.

Desenvolvedor Front-end, Catency (Freelance / PJ) — 04/2023 – 03/2025

- **Contexto:** Engenharia de landing pages e sites institucionais de alta conversão;
- **Arquitetura:** Desenvolvimento com Next.js e Tailwind CSS aplicando renderização híbrida (SSG/SSR) para controle de indexação (SEO) e otimização da entrega de estáticos. Auditorias Lighthouse contínuas para performance e acessibilidade (a11y/ARIA);
- **Impacto:** Entrega de 5 plataformas com LCP consistente < 2.5s e parâmetros de Core Web Vitals atingidos para alta indexação orgânica.

Desenvolvedor Full-stack, Umbler — 02/2022 – 04/2023

- **Contexto:** Criação de ferramentas internas para operação de Customer Success e gestão de portfólio;
- **Arquitetura:** Desenvolvimento ponta a ponta com .NET e Blazor sobre padrão MVC, separando modelagem de dados das interfaces de controle. Customização da biblioteca Bootstrap como Design System padronizado dos sistemas internos;
- **Impacto:** Viabilização da documentação de processos, trilhas de aprendizagem e automação da gestão de base de 100.000 clientes.

PROJETOS E CONTRIBUIÇÕES OPEN-SOURCE

Distributed Systems Lab

- Monorepo full-stack de alta performance: API de ingestão de webhooks (NestJS/Fastify, BullMQ), pipeline de processamento de dados em stream e dashboard analítico em tempo real (Next.js, Socket.io, Zustand);
- Processamento de 5 milhões de registros com 57MB de RAM (Node.js Streams + Postgres COPY); front-end a 60 FPS sob alta frequência de eventos WebSocket; API com 0% de taxa de erro e 7.35ms de latência (P95) sob 500 usuários simultâneos.

Guarahooks

- Biblioteca open-source com 50 React hooks, ~102 stars no GitHub, distribuída via registro shadcn e com CLI publicado no npm ([guara-cli](#)). Ecossistema de documentação integrado à content-collections;
- Manutenção de código consumido por aplicações de terceiros.

Enterprise React Architecture

- Implementação de referência para aplicações React de missão crítica. Abordagem "Shift-Left Testing" com TypeScript, Vitest e React Testing Library, aplicando TDD/BDD para validação de domínios complexos de UI;
- Modelo escalável que viabiliza pipelines de deploy com zero regressão.

MagicUI

- Desenvolvimento de 5 componentes com React.js, Tailwind CSS e Framer Motion. Lazy Loading e memorização (useMemo, useCallback) para otimização de bundle e ciclo de vida dos componentes.

EDUCAÇÃO

- **Pós-graduação em Engenharia de Software** — em andamento (início 2026);
- **Análise e Desenvolvimento de Sistemas** — Centro Universitário União das Américas Descomplica (2023 – 2025, concluído);
- **Informática para Internet** — Instituto Federal Sul-rio-grandense (2018 – 2022);
- **Formação complementar:** curso.dev (em andamento).

IDIOMAS

- Inglês: Avançado / Leitura Técnica Fluente
- Português: Nativo / Leitura e Escrita Técnica Fluente